

Hira Backend

API backend pour Hira — messagerie de l'écosystème MTAL.

Statut des tests : le routing, la validation des entrées, les middlewares (auth, erreurs, 404) et la logique métier pure ont été testés et validés. La connexion à une vraie instance MongoDB et l'upload réel vers S3/MinIO n'ont pas pu être testés dans l'environnement de développement (accès réseau restreint aux registres de paquets uniquement). Teste ces deux points en premier une fois en local — voir ci-dessous.

Stack

- Node.js + Express
- MongoDB (Mongoose)
- Socket.IO (temps réel)
- JWT (access + refresh token)

Démarrage

```
bash

# 1. Installer les dépendances
npm install

# 2. Copier le fichier d'environnement et le remplir
cp .env.example .env

# 3. Lancer MongoDB localement (ou utiliser MongoDB Atlas)
# Modifier MONGO_URI dans .env en conséquence

# 4. Démarrer en mode développement (redémarrage auto)
npm run dev

# Ou en production
npm start
```

L'API tourne par défaut sur `http://localhost:5000`.

Structure du projet

```
src/
├─ config/           # connexion base de données
├─ models/           # schémas Mongoose (User, ...)
├─ controllers/      # logique métier des routes
├─ routes/           # définition des endpoints
├─ middlewares/      # auth, gestion d'erreurs
├─ sockets/          # logique Socket.IO (temps réel)
├─ utils/            # fonctions utilitaires (tokens, asyncHandler)
├─ app.js            # configuration Express
└─ server.js         # point d'entrée (HTTP + Socket.IO)
```

Endpoints disponibles

Auth (`/api/auth`)

Méthode	Route	Description	Auth requise
POST	<code>/register</code>	Créer un compte	Non
POST	<code>/login</code>	Se connecter	Non
POST	<code>/refresh</code>	Renouveler l'access token	Non
POST	<code>/logout</code>	Se déconnecter	Oui

Body register : `{ name, email?, phone?, password }` (email ou phone requis) **Body login :** `{ identifiant, password }` (identifiant = email ou téléphone) **Body wuroen :** `{ wuroenToken }` — token JWT obtenu depuis une session Wuro'en active

Connexion via Wuro'en (écosystème MTAL)

Permet à un utilisateur déjà connecté sur Wuro'en de se connecter à Hira sans créer de nouveau mot de passe.

Fonctionnement

- Le client envoie le token JWT Wuro'en à `POST /api/auth/wuroen`
- Hira appelle l'API Wuro'en (`GET {WUROEN_API_URL}{WUROEN_ME_ENDPOINT}`) avec `Authorization: Bearer <token>`) pour vérifier le token et récupérer le profil
- Hira retrouve le compte lié (via `wuroenId`) ou le lie à un compte existant avec le même email, ou en crée un nouveau
- Hira émet ses propres tokens (access + refresh), comme pour une connexion classique

Configuration requise

env

WUROEN_API_URL=http://localhost:4000
WUROEN_ME_ENDPOINT=/api/auth/me

⚠ À ajuster impérativement

Le mapping des champs de réponse Wuro'en → Hira se trouve dans

`src/services/wuroenService.js` (fonction `verifyWuroenToken`). Le code suppose une réponse `{ id, name, email }` (ou `{ user: { id, name, email } }`) — si la vraie API Wuro'en renvoie une autre forme (ex: `fullName` au lieu de `name`), ajuste cette fonction en conséquence avant de tester en conditions réelles.

Statuts / Stories (`/api/status`) — toutes protégées par JWT

Statuts éphémères (texte ou média) visibles 24h, façon WhatsApp/Instagram. Expiration automatique gérée par un index TTL MongoDB — aucune tâche de nettoyage à programmer.

Méthode	Route	Description
GET	<code>/</code>	Statuts actifs des contacts + les miens, groupés par auteur
POST	<code>/</code>	Publie un statut : <code>{ type: "text" "image" "video", content?, backgroundColor?, media? }</code>
POST	<code>/:id/view</code>	Marque un statut comme vu (déclenche <code>status:viewed</code> en temps réel vers l'auteur)
DELETE	<code>/:id</code>	Supprime son propre statut avant expiration

Pour un statut média, uploade d'abord via `/api/media/upload` puis passe l'objet `media` retourné dans la création du statut — même flux que pour les messages.

Événements Socket.IO

Événement	Émis vers	Description
<code>status:new</code>	Contacts de l'auteur	Un contact vient de publier un statut
<code>status:viewed</code>	Auteur du statut	Quelqu'un a vu ton statut

Utilisateurs (/api/users) — toutes protégées par JWT

Méthode	Route	Description
GET	/me	Mon profil
PUT	/me	Modifier mon profil
GET	/search?q=...	Rechercher un utilisateur
GET	/contacts	Liste de mes contacts
POST	/contacts/:id	Ajouter un contact

Authentification des requêtes

Ajouter le header: Authorization: Bearer <accessToken>

Conversations (/api/conversations) — toutes protégées par JWT

Méthode	Route	Description
GET	/	Liste mes conversations (triées par activité récente)
POST	/private	Récupère ou crée une conversation privée: { userId }
POST	/group	Crée un groupe: { name, memberIds: [] }
GET	/:id/messages?before=&limit=	Historique paginé (curseur = id du plus ancien message chargé)
PUT	/:id	Modifie nom/avatar d'un groupe (admin uniquement): { name?, avatarUrl? }
POST	/:id/members	Ajoute des membres à un groupe (admin uniquement): { memberIds: [] }
DELETE	/:id/members/:userId	Retire un membre du groupe (admin uniquement)
POST	/:id/leave	Quitte un groupe (tout membre) — promeut automatiquement un nouvel admin si besoin

Chat temps réel (Socket.IO)

Authentification Socket.IO

Envoyer le token à la connexion :

```
js
```

```
const socket = io("http://localhost:5000", { auth: { token: accessToken } });
```

À la connexion, le serveur fait automatiquement rejoindre le client à toutes les rooms de ses conversations (une room = une conversation).

Événements émis par le client

Événement	Payload	Description
message:send	{ conversationId, type, content, media?, replyTo?, tempId }	Envoie un message. Callback renvoie { success, message, tempId }
typing:start	{ conversationId }	Signale que l'utilisateur écrit
typing:stop	{ conversationId }	Signale l'arrêt de la saisie
message:delivered	{ messageId }	Accuse réception d'un message
message:read	{ conversationId, messageId }	Marque un message comme lu
message:react	{ messageId, emoji }	Ajoute/change/retire (emoji null) une réaction
conversation:join	{ conversationId }	Rejoint une conversation créée après la connexion
message:edit	{ messageId, content }	Modifie un message texte (auteur uniquement). Callback { success, message }
message:delete	{ messageId }	Supprime un message (suppression douce, auteur uniquement). Callback { success }

Événements reçus par le client

Événement	Payload	Description
<code>message:new</code>	<code>{ message, tempId }</code>	Nouveau message dans une conversation rejointe
<code>typing:start</code> / <code>typing:stop</code>	<code>{ conversationId, userId }</code>	Un autre membre écrit / arrête
<code>message:delivered</code>	<code>{ messageId, userId }</code>	Un membre a reçu le message
<code>message:read</code>	<code>{ messageId, userId }</code>	Un membre a lu le message
<code>message:react</code>	<code>{ messageId, userId, emoji }</code>	Réaction ajoutée/modifiée
<code>message:edit</code>	<code>{ message }</code>	Un message a été modifié
<code>message:delete</code>	<code>{ messageId, conversationId }</code>	Un message a été supprimé
<code>user:status</code>	<code>{ userId, status }</code>	Un contact passe en ligne/hors ligne

Note sur `tempId` : le client génère un id temporaire côté UI pour afficher le message immédiatement (envoi optimiste), puis le remplace par le vrai message reçu via le callback ou l'événement `message:new`.

Upload média (S3-compatible)

Démarrer MinIO en local

```
bash
docker compose up -d
```

Console d'administration : <http://localhost:9001> (`hira_admin`) / (`hira_secret_key`)

Le serveur crée automatiquement le bucket (`S3_BUCKET`) au démarrage s'il n'existe pas.

Endpoint upload

Méthode	Route	Description
POST	<code>/api/media/upload</code>	Upload un fichier (<code>multipart/form-data</code> , champ <code>file</code>)
GET	<code>/api/media/signed-url?key=...</code>	URL signée temporaire (1h) pour un fichier privé

Réponse upload :

```
json
{
  "success": true,
  "category": "image",
  "media": {
    "key": "image/2026-08/uuid.jpg",
    "url": "http://localhost:9000/hira-media/image/2026-08/uuid.jpg",
    "fileName": "photo.jpg",
    "mimeType": "image/jpeg",
    "size": 234821
  }
}
```

Flux complet d'envoi d'un média dans un chat

1. Le client upload le fichier via `POST /api/media/upload` → reçoit `media` (url, key, etc.)
2. Le client envoie le message via Socket.IO (`message:send`) avec `type` et `media` dans le payload
3. Le message est diffusé aux autres membres via `message:new`, `media` inclus

Limites par catégorie

Catégorie	Types acceptés	Taille max
image	jpeg, png, webp, gif	10 Mo
video	mp4, mov, webm	100 Mo
audio	mp3, m4a, ogg, webm, wav	20 Mo
document	pdf, word, excel, txt, zip	50 Mo

Bucket privé vs public

Par défaut le service construit une URL directe (`buildPublicUrl`). Pour des conversations privées, il est recommandé de configurer le bucket en **privé** et de systématiquement passer par `/api/media/signed-url` pour générer une URL temporaire avant affichage côté client — évite que des médias privés soient accessibles par simple devinette d'URL.

Notifications push (Firebase Cloud Messaging)

Configuration

1. Créer un projet Firebase → Paramètres du projet → Comptes de service → Générer une nouvelle clé privée (télécharge un JSON)
2. Encoder ce fichier en base64 : `cat service-account.json | base64 -w 0`

3. Coller le résultat dans `FIREBASE_SERVICE_ACCOUNT` du `.env`

Sans cette variable, le serveur démarre normalement mais les push sont simplement désactivés (log d'avertissement, aucun crash).

Endpoints

Méthode	Route	Description
POST	<code>/api/users/push-token</code>	Enregistre le token FCM de l'appareil : <code>{ token, platform }</code> (<code>web</code>) (<code>android</code>) (<code>ios</code>)
DELETE	<code>/api/users/push-token</code>	Retire un token (à appeler à la déconnexion de l'appareil)

Comportement

- À chaque envoi de message (`message:send`), seuls les membres **non connectés en Socket.IO** à cet instant reçoivent un push — ceux déjà dans la conversation en direct reçoivent l'événement temps réel `message:new` uniquement, pas de double notification.
- Les tokens FCM devenus invalides (app désinstallée, etc.) sont automatiquement retirés de l'utilisateur après un envoi échoué.

Appels audio/vidéo 1-à-1 (Hira 2.0)

Signalisation WebRTC relayée via Socket.IO — le serveur ne transporte que les messages de négociation (offre/réponse SDP, candidats ICE), jamais le flux audio/vidéo lui-même qui passe en direct entre les deux navigateurs.

Événements Socket.IO

Événement	Payload	Description
<code>call:invite</code>	<code>{ toUserId, conversationId, offer, callType, fromName, fromAvatar }</code>	Initie un appel
<code>call:answer</code>	<code>{ toUserId, answer }</code>	Répond à un appel entrant
<code>call:ice-candidate</code>	<code>{ toUserId, candidate }</code>	Échange de candidats ICE
<code>call:reject</code>	<code>{ toUserId }</code>	Refuse un appel entrant
<code>call:end</code>	<code>{ toUserId }</code>	Raccroche

Limite connue

Utilise uniquement un serveur STUN public (Google) côté client — suffisant pour la majorité des connexions directes, mais certains réseaux très restrictifs (NAT symétrique, pare-feu d'entreprise strict) nécessiteront un serveur TURN pour relayer le flux média. Pas encore mis en place (coût d'infrastructure à prévoir).

Appels de groupe et partage d'écran (suite Hira 2.0)

Architecture **mesh** : chaque participant établit une connexion WebRTC directe avec chaque autre participant. Fonctionne bien jusqu'à 4-5 personnes ; au-delà, la charge réseau/CPU par appareil devient trop lourde (chaque participant encode et envoie son flux séparément vers chacun des autres). Un vrai SFU (serveur média) serait nécessaire pour aller au-delà.

État en mémoire

`activeGroupCalls` (Map `conversationId` → participants) vit dans le processus Node. **Limite** : ne fonctionne qu'avec une seule instance de serveur. Pour scaler horizontalement, cet état devra migrer vers Redis (comme la présence).

Événements Socket.IO

Événement	Payload	Description
<code>call:group:start</code>	<code>{ conversationId, callType, fromName, fromAvatar }</code>	Annonce un appel à tous les membres de la conversation
<code>call:group:join</code>	<code>{ conversationId, name, avatarUrl }</code> (callback)	Rejoint l'appel, reçoit la liste des participants déjà présents
<code>call:group:offer</code> / <code>answer</code> / <code>ice-candidate</code>	<code>{ toUserId, conversationId, ... }</code>	Négociation mesh ciblée entre deux participants
<code>call:group:leave</code>	<code>{ conversationId }</code>	Quitte l'appel (sans quitter la conversation)

Partage d'écran

Remplace la piste vidéo envoyée (`RTCTrpSender.replaceTrack`) par le flux `getDisplayMedia()` — fonctionne aussi bien en 1-à-1 qu'en groupe (dans ce cas, la piste est remplacée sur toutes les connexions mesh simultanément).

Prochaines étapes

- ☐ Serveur TURN pour les appels (réseaux restrictifs)
- ☐ SFU pour les appels de groupe au-delà de 5-6 participants
- ☐ Chiffrement de bout en bout
- ☐ Assistant IA, traduction instantanée (Hira 4.0)